



Multithreading & Synchronization

Concepts and Programming in C, Python, and Java

Learning outcomes

- 1 Understand threads vs processes and thread lifecycle, including the key differences in memory management, resource allocation, and execution context between these two fundamental concepts in operating systems.
- 2 Explain race conditions, mutexes, semaphores, and thread safety, with practical understanding of how synchronization mechanisms prevent data corruption in concurrent programming.
- 3 Apply concepts with examples in C, Python, and Java, demonstrating practical implementation of multithreading patterns across different programming languages.

Why multithreading?

Modern CPUs provide multiple cores, and multithreading helps exploit CPU parallelism and concurrency effectively. This allows applications to perform multiple tasks simultaneously, improving overall system performance and resource utilization.

Threads can perform various tasks such as UI responsiveness, background I/O operations, and parallel computations. This enables better user experience and efficient handling of resource-intensive operations.

Compared to multiple processes, threads are cheaper to create and switch because they share the same address space, reducing memory overhead and context switching costs.

Process vs Thread

Process

- Program in execution with its own separate address space.
- Possesses its own independent OS resources (file descriptors, memory).

Thread

- Lightweight execution unit inside a process.
- Shares code, data, and heap with peer threads.
- Maintains its own separate stack and CPU registers.

Implications of sharing

Threads can read and write the same variables directly, which enables fast communication between threads. This shared memory model allows threads to exchange data efficiently without the overhead of inter-process communication mechanisms.

However, unsynchronized access creates serious dangers: it can cause race conditions and inconsistent states when multiple threads modify shared data simultaneously. This is the primary source of concurrency bugs in multithreaded programs.

To ensure correctness, you must implement synchronization mechanisms to control access to critical sections. This involves using locks, mutexes, or other synchronization primitives to coordinate thread execution.

Basic multithreading model & scheduling

Single-process, multi-thread model

Multiple threads execute in the same virtual address space within a single process. This model allows threads to share memory and resources efficiently while maintaining separate execution contexts.

Ready

Running

Blocked

Terminated

Thread scheduling by OS

Threads are scheduled by the OS scheduler which determines when each thread gets CPU time. Each thread can be in states like ready, running, blocked, or terminated based on resource availability.

Concurrency and time slicing

Even on a single core processor, the OS switches between threads rapidly to create the illusion of simultaneous execution. This **time slicing** allows multiple threads to make progress within a single time quantum, with the OS managing context switches efficiently.

Context switch overhead: ~1-2 microseconds | Time quantum: 10-100ms

Thread lifecycle “conceptual”

States: New, Runnable (ready), Running, Blocked/Waiting, and Terminated. The thread transitions between these states based on scheduling and resource availability.

New → Runnable: Thread object created and start() (or equivalent) called. The thread is now ready to be scheduled by the OS.

Running → Blocked: Waiting for lock, I/O, or sleep; returns to Runnable when the condition is satisfied.

New

Runnable

Running

New → Runnable: Thread created, start() called. Runnable → Running: OS scheduler decides when to run.

Blocked /
Waiting

Running → Blocked: Thread waits for resource, lock, or I/O. Blocked → Runnable: Resource becomes available.

Terminated

Runnable → Terminated: Thread completes execution or is terminated. Cannot be restarted.

Thread lifecycle by language

Weâ€™ll map the conceptual lifecycle to Java, POSIX threads (C), and Python threading (with GIL specifics)

Java thread lifecycle

Java threads move through six states: NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, and TERMINATED. When a thread is created, it starts in the NEW state. Calling `start()` transitions it to RUNNABLE, where it becomes eligible for execution by the thread scheduler.

The scheduler decides when a RUNNABLE thread actually runs on the CPU. Threads may enter BLOCKED state when waiting for a monitor lock, WAITING when waiting indefinitely for another thread's notification, or TIMED_WAITING when waiting with a timeout.

When the `run()` method completes, the thread enters TERMINATED state and cannot be restarted. A terminated thread's resources are reclaimed by the JVM.

Java example “ creating and starting a thread

• • • Demo.java

```
class MyTask extends Thread {  
    public void run() {  
        System.out.println("Hello from " + Thread.currentThread().getName());  
    }  
}  
  
public class Demo {  
    public static void main(String[] args) {  
        Thread t = new MyTask(); // NEW  
        t.start();                // RUNNABLE → RUNNING  
    }  
}
```

C (pthreads) lifecycle & APIs

POSIX

POSIX threads (`pthread_t`) are native OS threads that run within a process. Their lifecycle closely parallels that of processes, with threads being scheduled by the operating system.

APIs

Key APIs include `pthread_create` for creating threads, `pthread_join` for waiting on thread completion, and `pthread_exit` for terminating threads.

Lifecycle

Threads follow a conceptual lifecycle where they can be created, runnable, blocked on synchronization or I/O operations, or terminated. The OS scheduler manages thread execution states.

C (pthreads) â€™ minimal example

```
demo.c

#include <pthread.h>
#include <stdio.h>

void* worker(void* arg) {
    printf("Hello from thread\n");
    return NULL;
}

int main() {
    pthread_t t;
    pthread_create(&t, NULL, worker, NULL);
    pthread_join(t, NULL); // wait for termination
    return 0;
}
```

Python threading & the GIL

In CPython, `threading.Thread` objects map to native OS threads, but the Global Interpreter Lock (GIL) ensures only one thread executes Python bytecode at a time. This is a mutex protecting Python's interpreter state that serializes execution of CPU-bound Python code across threads.

The GIL is released during blocking I/O operations, which means I/O-bound threads can run effectively and achieve true concurrency. However, for CPU-bound tasks, the GIL prevents multiple threads from executing Python bytecode simultaneously, making threading less effective for parallel computation.

For CPU-intensive work, consider using multiprocessing or alternative Python implementations like Jython or IronPython that don't have a GIL.

Python â€™ minimal example

demo.py

```
import threading

def worker():
    print("Hello from", threading.current_thread().name)

t = threading.Thread(target=worker)
t.start()
t.join()
```

Creating threads “ patterns

Across C, Python, and Java: define work, create thread, start, wait for completion

Creating threads “side-by-side” (C / Python / Java)

Notice the common pattern across languages: define work + start + join (wait)

```
C (pthreads)

// 1. Define target (worker
func)
// 2. Create and start
pthread_t t;
pthread_create(&t, NULL,
worker, NULL);

// 3. Wait for completion
pthread_join(t, NULL);
```

```
Python

# 1. Define target (worker
func)
# 2. Create thread
t =
threading.Thread(target=worker)

# 3. Start execution
t.start()

# 4. Wait for completion
t.join()
```

```
Java

// 1. Define target (Runnable)
// 2. Create thread
Thread t = new Thread(() -> {
    System.out.println("Hello");
});

// 3. Start execution
t.start();

// 4. Wait for completion
t.join();
```

All three implementations logically perform: Define Work → Create → Start → Wait (Join)

Shared data and critical sections

Shared data refers to variables, objects, files, or data structures that are accessed by multiple threads simultaneously. These shared resources are the primary source of concurrency issues when not properly managed.

Critical section is the part of code where shared variables are read or modified. The final result depends on the execution order of threads, making synchronization essential.

If multiple threads enter a critical section simultaneously without proper control, we may get inconsistent results due to race conditions.

Race conditions “definition”

A race condition is a concurrency bug that occurs when the result of executing a critical section depends on the relative timing or interleaving of threads. This happens when multiple threads access shared data concurrently and at least one thread performs a write operation.

Common patterns causing races: read“modify“write operations (increment, decrement), and check“then“act scenarios (checking if a value is not null before using it).

The goal of synchronization is to prevent data races by ensuring deterministic behavior regardless of thread scheduling. This requires proper locking mechanisms and careful design of critical sections.

Race condition “simple scenario

Scenario: Shared counter initially set to 0, with two threads each incrementing it 10,000 times.

Expected Result

20,000

(10,000 × 2)

Actual Result

Often < 20,000

(race condition)

Problem: The increment operation is not atomic. It consists of three steps: read current value, add 1, write back. When two threads interleave, both may read the same value, compute the same new value, and the last write "wins" - losing one increment.

Key Insight: The final result depends on the relative timing of thread execution - a classic race condition.

Race condition code “ C example

Ask students: run multiple times and observe non-deterministic output.

```
race_demo.c

#include <pthread.h>
#include <stdio.h>
int counter = 0; // shared global
void* increment(void* arg) {
    for (int i = 0; i < 100000; i++) {
        counter++; // NOT ATOMIC → race condition
    }
    return NULL;
}
int main() {    pthread_t t1, t2;
    pthread_create(&t1, NULL, increment, NULL);
    pthread_create(&t2, NULL, increment, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    printf("Counter = %d\n", counter); // often < 200000
    return 0;
}
```

Race condition code “ Python example

● ● ● race_condition.py

```
import threading
counter = 0
def increment():
    global counter
    for _ in range(100000):
        counter += 1 # read-modify-write, not atomic
threads = [threading.Thread(target=increment) for _ in range(2)]
for t in threads: t.start()
for t in threads: t.join()
print("Counter =", counter)
```

Note: Even with the GIL, Python’s counter += 1 is not guaranteed to be atomic across threads.

Race condition code “ Java example

```
RaceDemo.java

class Counter {
    public int value = 0;
}

public class RaceDemo {
    public static void main(String[] args) throws InterruptedException {
        Counter c = new Counter();
        Runnable task = () -> {
            for (int i = 0; i < 100000; i++) c.value++; // not atomic
        };
        Thread t1 = new Thread(task), t2 = new Thread(task);
        t1.start(); t2.start();
        t1.join(); t2.join();
        System.out.println("Counter = " + c.value);
    }
}
```

Note: Java™s `c.value++` is a non-atomic read-modify-write.

Synchronization “goals & techniques”

Goals: Synchronization aims to achieve mutual exclusion (only one thread in critical section), establish ordering constraints between threads, and ensure visibility of updates across threads so that changes made by one thread are visible to others.

Techniques:

- Locks/mutexes (mutual exclusion objects)
- Semaphores (counting and binary)
- Condition variables / wait-notify, barriers
- Atomic variables and lock-free algorithms

Mutex “concept”

A mutex is a mutual exclusion object used to protect a critical section so that only one thread at a time can execute it.

Mutexes follow a locking mechanism: a thread acquires (locks) before entering and releases (unlocks) when leaving the critical section.

Typically, the same thread that locks must unlock the mutex; other threads attempting to lock will block until it is unlocked.

This ensures that shared resources are accessed in a controlled manner, preventing race conditions and data corruption.

Semaphore “concept”

A semaphore is an integer variable used as a signaling mechanism to control access to a resource pool. It maintains a count that represents the number of available resources.

wait (P/down): decrements the semaphore. If the value becomes negative, the calling thread is blocked until the semaphore is incremented by another thread.

signal (V/up): increments the semaphore. If there are waiting threads, one of them is unblocked and allowed to proceed.

Types: counting semaphore (value represents number of available resources) and binary semaphore (value is 0 or 1, similar to mutex semantics).

Mutex vs Semaphore “comparison

Aspect	Mutex	Semaphore
Mechanism	Locking	Signaling
Type	Object	Integer counter variable
Ownership	Typically only owner thread unlocks	Any thread may signal
Count	Only one owner at a time	Counting or binary; can allow multiple simultaneous holders
Use Case	Exclusive access to a single shared resource	Limit concurrent access to N instances (e.g., N connections)

Mutex example “ C (pthread)

```
increment.c

#include <pthread.h>
#include <stdio.h>

int counter = 0;
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

void* increment(void* arg) {
    for (int i = 0; i < 100000; i++) {
        pthread_mutex_lock(&lock);
        counter++;
        pthread_mutex_unlock(&lock);
    }
    return NULL;
}
```

Mutex example “ Java (synchronized)

SafeCounter.java

```
class SafeCounter {  
    private int value = 0;  
    public synchronized void inc() { value++; } // protected critical  
section  
    public int get() { return value; }  
}
```

Note: `synchronized` methods/blocks provide mutual exclusion and visibility guarantees.

Semaphore example in C (POSIX)

Pattern: limit concurrency to available slots.

```
worker.c

#include <semaphore.h>

sem_t slots;

void* worker(void* arg) {
    sem_wait(&slots);    // acquire a slot
    // do work
    sem_post(&slots);    // release slot
    return NULL;
}
```

Semaphore example “ Java

SemaphoreDemo.java

```
import java.util.concurrent.Semaphore;

Semaphore slots = new Semaphore(3); // at most 3 concurrent workers

Runnable task = () -> {
    try {
        slots.acquire();
        // limited resource work
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    } finally {
        slots.release();
    }
};
```

Python synchronization â€” Lock and Semaphore

sync_demo.py

```
import threading
counter = 0
lock = threading.Lock()
def increment():
    global counter
    for _ in range(100000):
        with lock:          # acquire, then release
            counter += 1

sem = threading.Semaphore(3)

def limited_task():
    with sem:               # at most 3 such functions running simultaneously
        # do limited work
    pass
```

Deadlocks, livelocks, starvation

Deadlock is a situation where a set of threads becomes permanently blocked because each thread is waiting for a resource held by another thread in the same set, creating a circular wait condition that prevents any progress.

Livelock occurs when threads keep changing state in response to each other but never actually make progress. This is different from deadlock as the threads are not blocked, but they are stuck in a loop of yielding to each other.

Starvation happens when a thread never gets CPU time or access to a lock because other threads consistently beat it to the resource, often due to unfair scheduling algorithms.

Prevention tip: To avoid deadlocks, always acquire multiple locks in a consistent global order across all threads, which prevents circular wait conditions.

Thread-safe programming & strategies

Thread-safe code behaves correctly even when multiple threads execute it concurrently on shared data. The key practical notions of safety include having no data races or inconsistent states, ensuring deterministic behavior independent of thread interleaving for specified preconditions, and providing both mutual exclusion and visibility guarantees.

Avoid sharing

Use thread-local variables or pass copies instead of sharing objects.

Immutability

Immutable objects can be freely shared across threads without locks.

Confinement

Restrict object access to a single thread (e.g., UI thread confinement).

When sharing mutable state

Use locks, atomic variables, or specialized thread-safe collections/abstractions to ensure proper synchronization.

Higher-level concurrency utilities

Java

The `java.util.concurrent` package provides comprehensive concurrency utilities including thread pools (`ExecutorService`), concurrent collections, locks (`ReentrantLock`), and atomic variables (`AtomicInteger`).

Python

The `concurrent.futures.ThreadPoolExecutor` encapsulates worker threads and task submission, providing a high-level interface for managing thread pools.

C (Libraries)

Abstraction is often library-specific (e.g., `OpenMP` , `TBB`) built on top of pthreads, providing higher-level constructs for parallel programming.

```
ExecutorService pool = Executors.newFixedThreadPool(4);  
pool.submit(() -> doWork());  
pool.shutdown();
```

Memory visibility: GIL vs Java Memory Model

CPython GIL: protects interpreter state; does not make compound operations on objects logically atomic.

Java Memory Model: defines visibility and order; synchronized, volatile, and final provide happens before guarantees.

Without visibility guarantees, threads may read stale data even if no race seems apparent.

Debugging concurrency + best practices

Symptoms

Intermittent failures, nonâ€™reproducible outputs, hangs, and data corruption are common indicators of concurrency bugs. These issues often appear randomly and are difficult to reproduce consistently.

Tools & Techniques

Logging with thread IDs, IDE debuggers/profilers, static analyzers, and using small test loads to reproduce race conditions are effective debugging approaches.

Best Practices

Minimize shared state; use thread pools instead of raw threads. Keep critical sections small and clearly define resource ownership.

Document invariants; avoid nested locks or acquire in fixed order to prevent deadlocks.

Producerâ€™Consumer (Bounded Buffer)

We will implement producerâ€™consumer using the following approaches:

For C, we will use pthreads with mutex and condition variables to create a thread-safe circular buffer. This approach demonstrates low-level synchronization primitives.

For Java, we will implement synchronized methods with wait and notify to handle producer and consumer coordination on a shared buffer.

For Python, we will explore both queue.Queue for high-level abstractions and Condition variables for low-level control.

Producerâ€™Consumer in C (pthreads) â€™ part 1

```
producer_consumer_pthreads.c

#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#define BUFFER_SIZE 5
int buffer[BUFFER_SIZE];
int in = 0, out = 0, count = 0;
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t not_full = PTHREAD_COND_INITIALIZER;
pthread_cond_t not_empty = PTHREAD_COND_INITIALIZER;
void* producer(void* arg) {
    while (1) {
        int item = rand() % 100; // produce an item
        pthread_mutex_lock(&mutex);
        while (count == BUFFER_SIZE) {
            pthread_cond_wait(&not_full, &mutex);
        }
        buffer[in] = item;
        in = (in + 1) % BUFFER_SIZE;
        pthread_cond_signal(&not_empty);
        sleep(1); // simulate production time
        printf("Produced: %d at %d\n", item, in);
        count++;
        pthread_mutex_unlock(&mutex);
    }
    return NULL;
}
```

Producerâ€™Consumer in C (pthread) â€™ part 2

```
producer_consumer.c

void* consumer(void* arg) {
    while (1) {
        pthread_mutex_lock(&mutex);
        while (count == 0) {
            pthread_cond_wait(&not_empty, &mutex);
        }
        int item = buffer[out];
        printf("Consumed: %d at %d\n", item, out);
        out = (out + 1) % BUFFER_SIZE;
        count--;
        pthread_cond_signal(&not_full);
        pthread_mutex_unlock(&mutex);
        sleep(1);
    }
    return NULL;
}

int main(void) {
    pthread_t prod, cons;
    pthread_create(&prod, NULL, producer, NULL);
    pthread_create(&cons, NULL, consumer, NULL);
    pthread_join(prod, NULL);
    pthread_join(cons, NULL);
    pthread_mutex_destroy(&mutex);
    pthread_cond_destroy(&not_full);
    pthread_cond_destroy(&not_empty);
    return 0;
}
```

Key points: Buffer indices/count are protected by a mutex. `not_full` and `not_empty` condition variables implement blocking behavior instead of busyâ€™waiting.

Producer-Consumer in Java (wait/notify) part 1

● ● ● ProducerConsumer.java

```
// ProducerConsumer.java (excerpt)
import java.util.LinkedList;
import java.util.Queue;
public class ProducerConsumer {
    private static final int CAPACITY = 5;
    private final Queue<Integer> buffer = new LinkedList<>();
    public static void main(String[] args) {
        ProducerConsumer pc = new ProducerConsumer();
        Thread producer = new Thread(pc.new Producer(), "Producer");
        Thread consumer = new Thread(pc.new Consumer(), "Consumer");
        producer.start(); consumer.start();
    }
    class Producer implements Runnable {
        private int value = 0;
        public void run() {
            try { while (true) { produce(value++); Thread.sleep(500); } }
            catch (InterruptedException e) { Thread.currentThread().interrupt(); }
        }
        private void produce(int item) throws InterruptedException {
            synchronized (buffer) {
                while (buffer.size() == CAPACITY) { buffer.wait(); }
                buffer.add(item);
                System.out.println("Produced: " + item);
                buffer.notifyAll();
            }
        }
    }
}
```

Producerâ€™Consumer in Java (wait/notify) â€™ part 2

```
Consumer.java

class Consumer implements Runnable {
    public void run() {
        try {
            while (true) {
                int item = consume();
                System.out.println("Consumed: " + item);        Thread.sleep(800);
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
    private int consume() throws InterruptedException {
        synchronized (buffer) {
            while (buffer.isEmpty()) { buffer.wait(); }
            int item = buffer.remove();        buffer.notifyAll();
            return item;
        }
    }
}
```

Key points:wait() / notifyAll() must be called inside synchronized on the same monitor; whileâ€™loops handle spurious wakeups.

Producerâ€™Consumer in Python â€™ Queue variant (part 1)

```
producer_consumer_queue.py

# producer_consumer_queue.py (excerpt)
import threading, time, random
from queue import Queue
BUFFER_SIZE = 5
queue = Queue(maxsize=BUFFER_SIZE)
def producer():
    item = 0
    while True:
        time.sleep(random.uniform(0.2, 1.0))
        queue.put(item) # blocks if queue is full
        print(f"Produced: {item} (size={queue.qsize()})")
        item += 1
```

Producerâ€™Consumer in Python â€™ Queue variant (part 2)

```
producer_consumer_queue.py

def consumer():
    while True:
        item = queue.get() # blocks if queue is empty
        print(f"Consumed: {item} (size={queue.qsize()})")
        time.sleep(random.uniform(0.5, 1.2))
        queue.task_done()

if __name__ == "__main__":
    t_prod = threading.Thread(target=producer, daemon=True)
    t_cons = threading.Thread(target=consumer, daemon=True)
    t_prod.start(); t_cons.start()
    time.sleep(10)
    print("Main thread exiting...")
```

Note: Queue is threadâ€™safe; handles locking and signaling internally. See also Conditionâ€™based variant for OSâ€™theory alignment.

Producerâ€™Consumer in Python â€™ Condition variant

Shared Setup

Shared buffer of size N with `in_idx` for next free position, `out_idx` for next full position, and `count` for tracking items. Initialize `mutex = Lock()`, `not_full = Condition(mutex)`, and `not_empty = Condition(mutex)`.

Producer Flow with `not_full`

```
with not_full:
    while count == N: not_full.wait()
    buffer[in_idx] = item; in_idx = (in_idx + 1) % BUFFER_SIZE
    count += 1; not_empty.notify()
```

Consumer Flow with `not_empty`

```
with not_empty:
    while count == 0: not_empty.wait()
    item = buffer[out_idx]; out_idx = (out_idx + 1) % BUFFER_SIZE
    count -= 1; not_full.notify()
```

Note:

This mirrors C pthreads logic with condition variables, making it ideal for bounded buffer semantics and OS theory alignment.

Wrap-up & exam-style questions

Recap

Threads vs processes and thread lifecycle across Java, C (pthreads), and Python.

Race conditions, critical sections, and the need for controlled access to shared state.

Synchronization mechanisms using **mutexes** and **semaphores**.

Basics of **thread-safe design** and avoiding deadlocks.

Practical **producer-consumer** implementations.

Sample Questions

1. Explain, with a diagram, the Java thread lifecycle, including the states and methods that trigger transitions.
2. Give a code example of a race condition and demonstrate how to fix it using a mutex in a language of your choice.
3. Compare and contrast a mutex and a semaphore. Provide an example scenario where a counting semaphore is preferred over a mutex.